Name: Phan Tran Thanh Huy

ID: ITCSIU22056

OS LAB 10

1. What happens if you turn on the parallelism flag (-p)? How much would you expect performance to change when each thread is working on adding different vectors (which is what -p enables) versus working on the same ones?

2. Now let's study vector-try-wait.c. First make sure you understand the code. Is the first call to pthread_mutex_trylock() really needed? Now run the code. How fast does it run compared to the global order approach? How does the number of retries, as counted by the code, change as the number of threads increases?

3. Now let's look at vector-avoid-hold-and-wait.c. What is the main problem with this approach? How does its performance compare to the other versions, when running both with -p and without it?

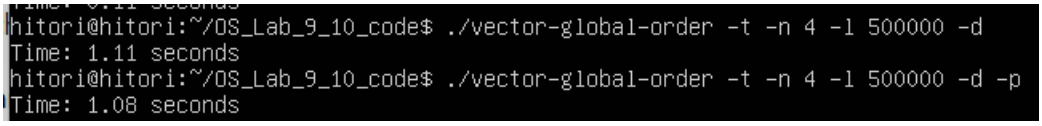
4. Finally, let's look at vector-nolock.c. This version doesn't use locks at all; does it provide the exact same semantics like the other versions? Why or why not?

5 .Now compare its performance to the other versions, both when threads are working on the same two vectors (no -p) and when each thread is working on separate vectors (-p). How does this no-lock version perform?

1.

When the -p flag enabled, the total execution time is expected to decrease, and the performance should scale much better as the number of threads or the number of loops increases. The improvement becomes more pronounced as contention would otherwise increase in the non-parallel case.

Reason: Each thread performs vector additions on different vectors instead of all threads operating on the same pair of vectors. This significantly reduces lock contention because threads no longer compete for the same mutexes. As the tests below



2.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "mythreads.h"

#include "main-header.h"
#include "vector-header.h"

int retry = 0;

void vector_add(vector_t *v_dst, vector_t *v_src) {
  top:
    if (pthread_mutex_trylock(&v_dst->lock) != 0) {
    goto top;
    }
    if (pthread_mutex_trylock(&v_src->lock) != 0) {
    retry++;
    Pthread_mutex_unlock(&v_dst->lock);
    goto top;
    }
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
    v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    Pthread_mutex_unlock(&v_dst->lock);
    Pthread_mutex_unlock(&v_src->lock);
}

void fini() {
    printf("Retries: %d\n", retry);
}

#include "main-common.c"
```

Is the first call to pthread_mutex_trylock() really needed?
Yes, it is needed. It attempts to acquire the first lock without blocking, enabling the try-wait mechanism to avoid deadlocks.

Instead of blocking on the first lock (like Pthread_mutex_lock), we try to acquire it. If it fails, we immediately retry (goto top).

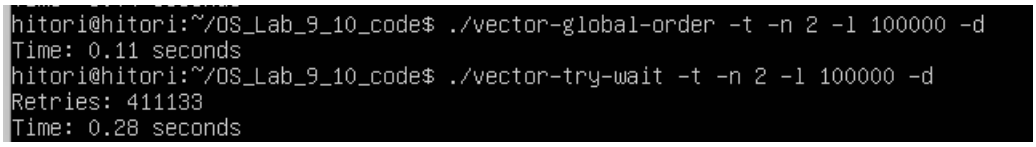
- This is how the code can avoid classic deadlocks: threads won't get stuck waiting for locks in a fixed order; they back off and retry.
- Without it, the code would just block on the first lock, which could still cause contention, but it would be simpler.

How fast does it run compared to the global order approach?

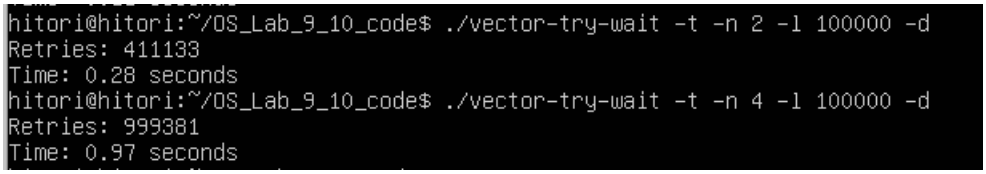
- vector-global-order.c ensures deadlock-free execution by always acquiring locks in a fixed order **(v_dst < v_src).**
- vector-try-wait.c ensures deadlock-free execution by retrying if the second lock is unavailable.

Performance difference:

- For **low thread count and small workloads**, the time is **similar**, because retries rarely happen.
- For **many threads and high contention**, vector-try-wait.c may **run slower**, because threads sometimes retry (busy-waiting increases CPU usage).
- It runs similarly for a small number of threads or low contention. With more threads or higher contention, it may be slower due to retries.



How does the number of retries, as counted by the code, change as the number of threads increases?



The number of retries increases as the number of threads increases, because more threads contend for the same locks.

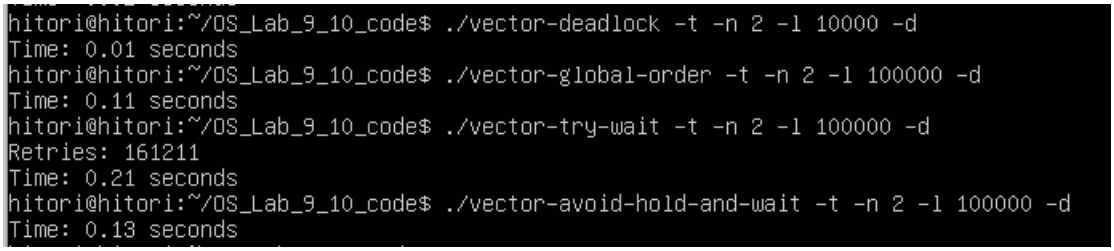
3.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <unistd.h>

#include "mythreads.h"

#include "main-header.h"
#include "vector-header.h"

// use this to make lock acquisition ATOMIC
pthread_mutex_t global = PTHREAD_MUTEX_INITIALIZER;

void vector_add(vector_t *v_dst, vector_t *v_src) {
    // put GLOBAL lock around all lock acquisition...
    Pthread_mutex_lock(&global);
    Pthread_mutex_lock(&v_dst->lock);
    Pthread_mutex_lock(&v_src->lock);
    Pthread_mutex_unlock(&global);
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
    v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    Pthread_mutex_unlock(&v_dst->lock);
    Pthread_mutex_unlock(&v_src->lock);
}

void fini() {}

#include "main-common.c"
```

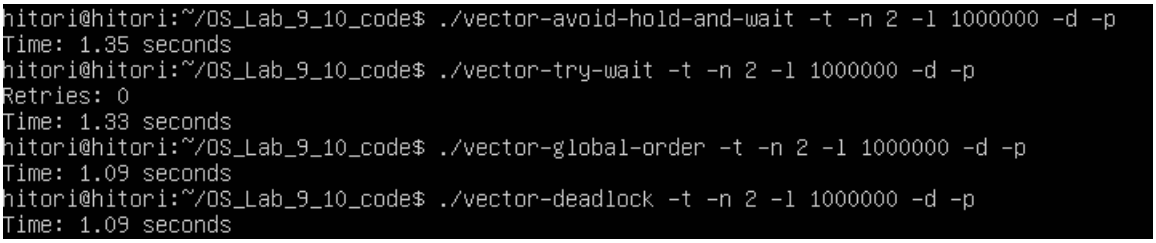
Main problem:

The global lock (global) serializes all vector additions, so only one thread at a time can acquire any vector locks. This essentially eliminates parallelism and reduces concurrency, defeating the purpose of fine-grained locking.

- **Without -p**



- Performance is safe but might be slower than global-order or try-wait approaches due to extra locking overhead.
- **With -p**



- Performance is worse than other versions because the global lock forces threads to wait unnecessarily, even when they could safely operate on separate vectors.

4. The vector-nolock.c version does not use mutexes at all and instead relies on the atomic fetch_and_add() instruction.

As the semantics, It does not provide exactly the same semantics as the locked versions. While fetch_and_add() ensures that each individual addition is atomic, there is no overall atomicity for the whole vector operation.

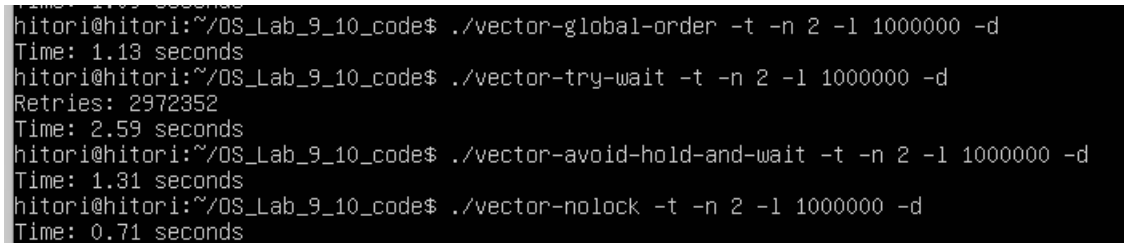
- Another thread could interleave operations on other elements of the vector, leading to results that differ from a fully locked vector_add().

Reason: In locked versions, the entire vector_add() operation is protected by mutexes, so no other thread can modify either vector during the operation. In vector-nolock.c, threads can still interfere element by element, so the final vector may reflect partially interleaved additions.

- It's faster but may produce slightly different results under heavy parallelism.

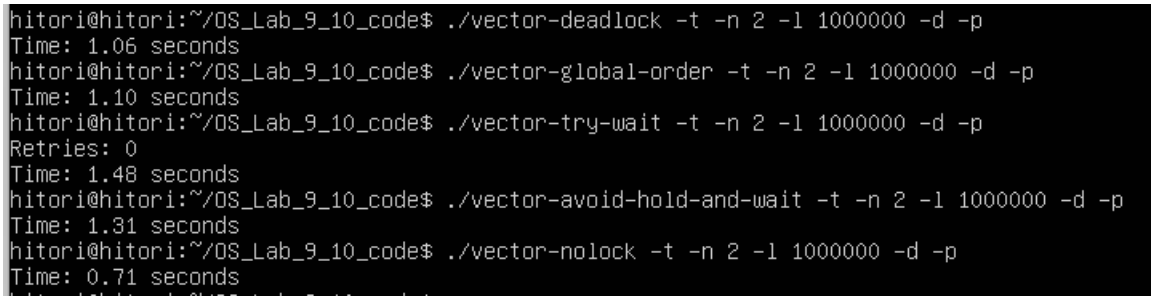
5. 3

- **Without -p**



➔ The performance is high because threads are not blocked by locks. However, It can still have race conditions, meaning the results may not be correct.

➔ **With -p**



➔ Performance is excellent and close to ideal parallel speed, because there's no contention between threads. Also, Results are correct in this case, since no two threads touch the same memory.